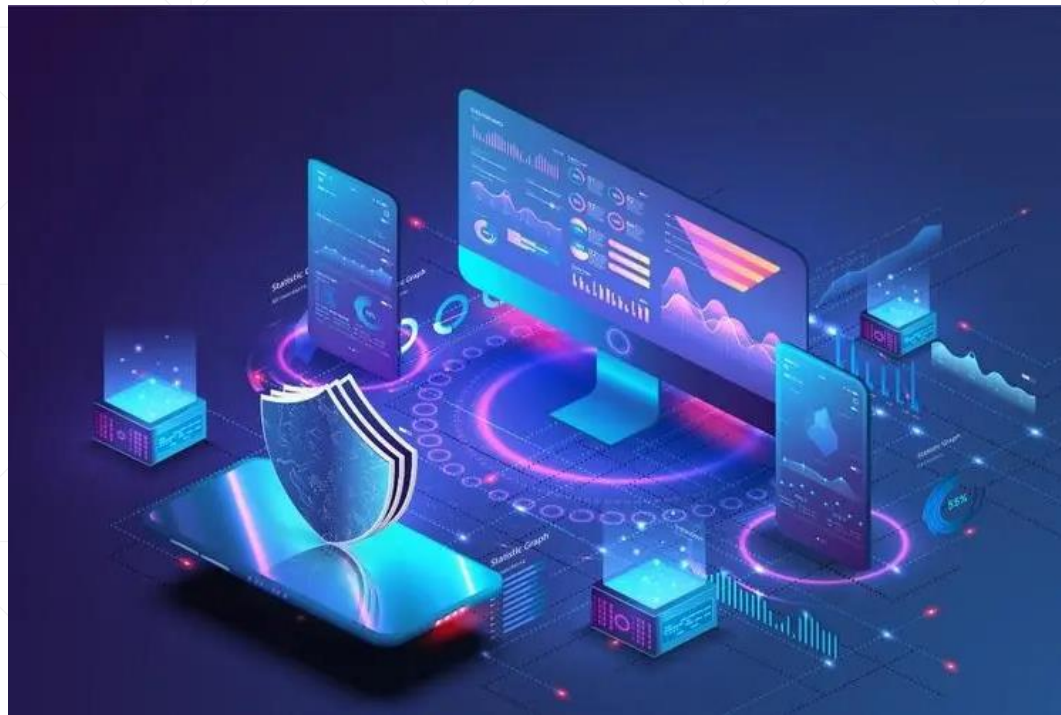




ThreadLocal应用实例

北京理工大学计算机学院
金旭亮

应用实例

在实际开发中，线程池非常常见。它适合于批量执行特定的数据处理任务。

如果将`ThreadLocal`与线程池中的线程结合起来，为每个线程都提供一个独享的对象，这样一来，无需考虑线程同步问题，访问此对象无需加锁，对象中的方法也不需要设计为“线程安全的”，还不用担心对象创建过多消耗太多的资源（因为线程池中的线程个数是有限的），很明显是个划算的买卖。

下面我们设计一个实例，创建一个包容两个线程的线程池，每个线程配置一个`Worker`对象，然后，向线程池提交1000个任务.....

示例分析

```
public class Worker {  
    4 usages  
    private String name;  
    public String getName() {...}  
    public void setName(String name) {...}  
    1 usage  
    public Worker(String name) {  
        this.name = name;  
    }  
    1 usage  
    public void process(String task) throws InterruptedException {  
        System.out.println(name+"正在处理"+task);  
        Thread.sleep(millis: 1000);  
        System.out.println(task+"处理完毕。");  
    }  
}
```

//为线程池中的每个线程，提供一个“独享”的工作对象

```
public class UseThreadPool {
```

//用于为Worker实例生成唯一的Id值

1 usage

```
private static AtomicInteger workerCount = new AtomicInteger();
```

```
public static void main(String[] args) {
```

//创建一个包容两个线程的线程池

```
ExecutorService threadPool = Executors.newFixedThreadPool(nThreads: 2);
```

//每次get，都会得到一个新的Worker实例

```
ThreadLocal<Worker> workers = ThreadLocal.withInitial(() -> {  
    return new Worker(name: "Worker" + workerCount.getAndIncrement());  
});
```

//创建1000个待处理的任务

```
for (int i = 0; i < 1000; i++) {...}
```

//关闭线程池

```
threadPool.shutdown();
```

```
}
```

```
}
```

代码见下页

//创建一个包容两个线程的线程池

```
ExecutorService threadPool = Executors.newFixedThreadPool(nThreads: 2);
```

//每次get, 都会得到一个新的Worker实例

```
ThreadLocal<Worker> workers = ThreadLocal.withInitial(() -> {  
    return new Worker(name: "Worker" + workerCount.getAndIncrement());  
});
```

//创建1000个待处理的任务

```
for (int i = 0; i < 1000; i++) {  
    int finalI = i;  
    Runnable runnable = () -> {  
        try {  
            //分配一个Worker实例完成特定的工作任务  
workers.get().process(task: "任务" + finalI);  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    };  
    //将任务提交给线程池  
    threadPool.submit(runnable);  
}
```

这里的关键点在于, Worker实例并不等于1000, 而是与线程池中的线程一一对应, 在本例中, 不过2个而已。

```
Run: UseThreadPool x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
Worker1正在处理任务1  
Worker0正在处理任务0  
任务1处理完毕。  
任务0处理完毕。  
Worker0正在处理任务2  
Worker1正在处理任务3  
任务3处理完毕。
```

实例二：

有一些信息，如果把它设置为全局变量，会给代码编写带来方便（比如，就不要为方法声明特定的参数，而是在方法体内直接写代码访问全局变量），这时，就可以考虑使用`ThreadLocal`让这些变量可以跨线程地安全访问。为此，设计以下实例：

同一个线程调用的多个方法，存取同一个`ThreadLocal`中的数据，由于是顺序执行，所以无需加锁。

启动多个上述线程，它们互不影响，“各回各家，各找各妈”。

//封装线程工作所需要存取的信息项

2 usages

```
class MyInfo {  
    3 usages  
    private String info;  
    6 usages  
    public String getInfo() {...}  
    3 usages  
    public void setInfo(String info) {...}  
    1 usage  
    public MyInfo(String info) {  
        this.info = info;  
    }  
}
```

//供每个线程独享的数据

3 usages

```
class MyInfoHolder {  
    3 usages  
    public static ThreadLocal<MyInfo> myInfoInThreadLocal =  
        ThreadLocal.withInitial(() ->  
            new MyInfo(""));  
}
```

三个需要存取数据的类型

```
class VisitorZhangShan {  
    1 usage  
    void visit() {  
        var myInfo : MyInfo = MyInfoHolder.myInfoInThreadLocal.get();  
        myInfo.setInfo(myInfo.getInfo() + "张三到此一游");  
        System.out.println(myInfo.hashCode() + ":" + myInfo.getInfo());  
    }  
}
```

还有另两个“李四”和“王五”类，功能与代码都是类似的。


```
public class GlobalInfoDemo {  
    public static void main(String[] args) {  
        //让三个类顺序存取MyInfo对象  
        Runnable runnable = () -> {  
            new VisitorZhangShan().visit();  
            new VisitorLiShi().visit();  
            new VisitorWangWu().visit();  
        };  
        //三个线程并行运行  
        for (int i = 0; i < 3; i++) {  
            new Thread(runnable).start();  
        }  
    }  
}
```

Run: GlobalInfoDemo x

"C:\Program Files\Java\jdk-17\bin\java.exe"

2058282013:张三到此一游
1429075913:张三到此一游
757218850:张三到此一游
757218850:张三到此一游 李四到此一游
2058282013:张三到此一游 李四到此一游
1429075913:张三到此一游 李四到此一游
2058282013:张三到此一游 李四到此一游 王五到此一游
1429075913:张三到此一游 李四到此一游 王五到此一游
757218850:张三到此一游 李四到此一游 王五到此一游

Process finished with exit code 0

小结



使用ThreadLocal有以下好处：



通过隔离性可以实现线程安全



不需要加锁，提高执行效率